



Penetration Testing

Rubicon (Digital Asset)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	6
Security Rating	7
Severity Definitions	8
Status Definitions	9
Penetration Test Findings	10
Conclusion	17
Our Methodology	18
Disclaimers	20
About Hashlock	21

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Rubicon team partnered with Hashlock to conduct a security Penetration Test of their Rubicon Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Project Rubicon is a novel cross-chain staking infrastructure purpose-built for the Bittensor ecosystem. It enables users to stake subnet-native tokens (Alpha) to earn validator rewards while simultaneously unlocking liquidity via xAlpha tokens (representing staked Alpha).

This project is spearheaded by General Tao Ventures in conjunction with Chainlink, who intend for the Rubicon to become the canonical staking solution for the TAO ecosystem.

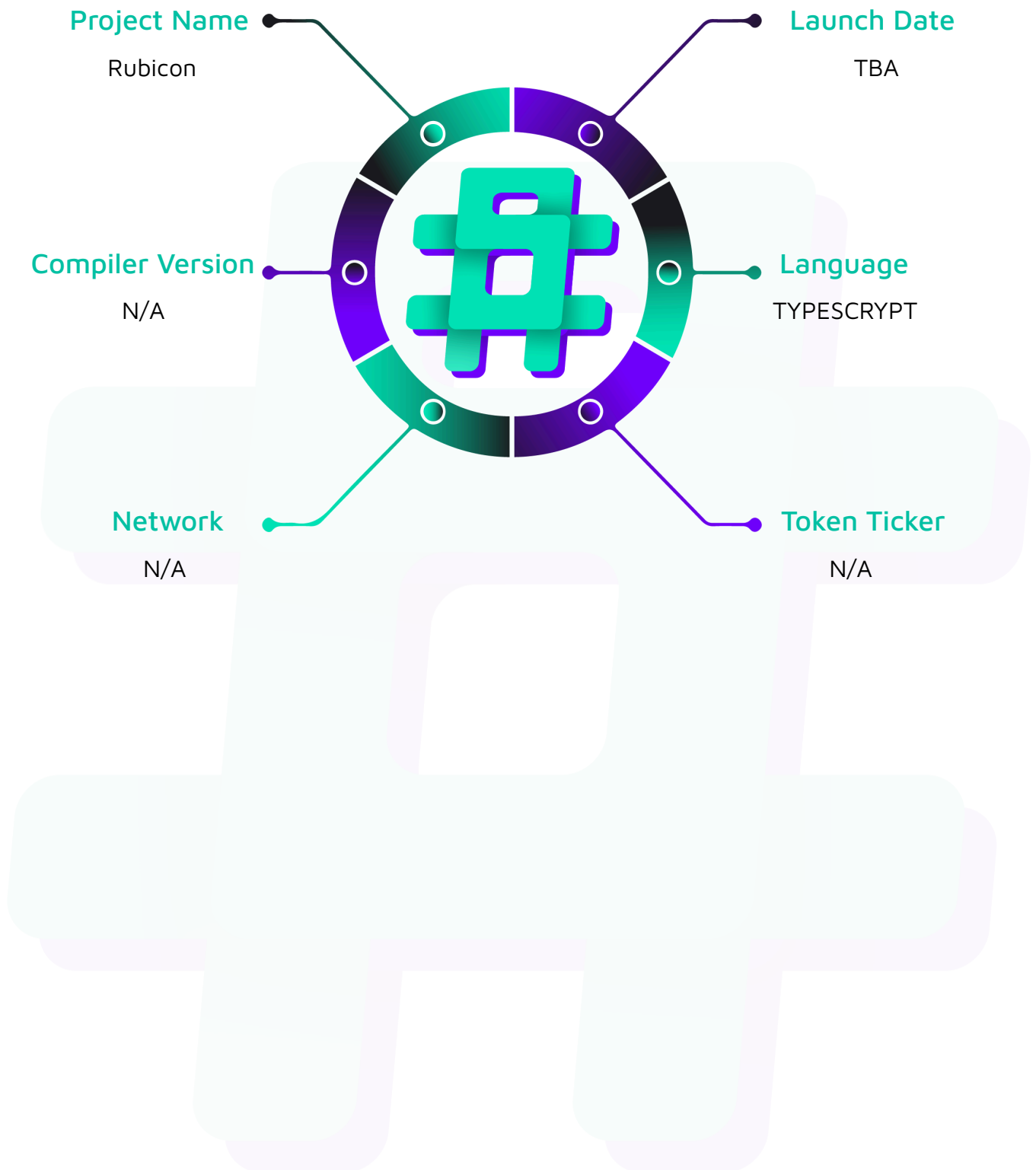
Project Name: Rubicon

Project Type: Digital Asset, RWA, Security Tokens, RWAfi, TradFi, DeFi

Website: <https://www.rubiconbridge.io/>

Logo:

PROJECT
RUBICON

Visualised Context:

Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Rubicon project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Rubicon Code Base
Platform	Web apps & APIs
Penetration Test Date	October, 2025
Repository/Website URL/IP Tested	https://github.com/General-Tao-Ventures/lsa-front-end
GitHub Commit Hash (If applicable)	64b80e54f856cde7c9661c8f6c00a923f1096d86
Fix Review GitHub Commit Hash (If applicable)	aca1c95a13e1b94f5308c7b2feca8e999b3bd96d

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 High-severity vulnerabilities

2 Medium-severity vulnerabilities

Caution: Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] Infura API Key Exposure - Public API key embedded in client bundle causing financial abuse risk

Description

The Infura API key is exposed in the client-side JavaScript bundle through the `NEXT\_PUBLIC\_INFURA\_API\_KEY` environment variable, allowing attackers to extract and abuse the API key for unauthorized requests, leading to quota exhaustion and unexpected billing charges.

Vulnerability Details

The `NEXT\_PUBLIC\_INFURA\_API\_KEY` environment variable is embedded in the client bundle and used to construct RPC URLs:

```
pages/_app.tsx line=8

const infuraApiKey = process.env.NEXT_PUBLIC_INFURA_API_KEY;

pages/_app.tsx start=29

new WalletLinkConnector({
  options: {
    appName: "NextJS-wagmi",
    jsonRpcUrl: `${rpcUrl}/${infuraApiKey}`, // API key visible in URL
  },
}),
```

Next.js automatically embeds any environment variable prefixed with `NEXT_PUBLIC_` into the client-side JavaScript bundle, making it accessible to anyone who inspects the compiled code.

Impact

Attackers can consume the project's Infura quota, leading to unexpected costs, service disruption due to wallet connection failures for legitimate users, resource abuse by other applications using the API key, and significant unexpected charges from sustained attacks.

Recommendation

1. Remove the `'NEXT_PUBLIC_'` prefix from the environment variable.
2. Create a server-side API route to proxy Infura requests.

Status

Resolved

[H-02] Severely Outdated Dependencies - Critical security vulnerabilities in 3+ year old packages

Description

The application uses severely outdated dependencies that are 3+ years behind current versions, containing multiple known security vulnerabilities, including Next.js 12.0.7, React 17.0.2, and other critical packages.

Vulnerability Details

```
package.json start=11

"dependencies": {

  "@headlessui/react": "^1.4.2",    // 3+ years old

  "next": "12.0.7",                 // Released Dec 2021 - CRITICAL CVEs

  "react": "17.0.2",                // Released Mar 2021 - Missing security patches

  "react-dom": "17.0.2",            // Released Mar 2021

  "react-dropdown": "^1.9.2",       // Unmaintained package

  "react-icons": "^4.3.1",          // Missing updates

  "wagmi": "^0.1.2"                 // Very early version with vulnerabilities

}
```

These versions contain documented security vulnerabilities and are missing years of critical security patches.

Impact

Next.js 12.x has documented XSS and build-time vulnerabilities, creating supply chain risk due to vulnerable dependencies, increasing maintenance debt, and resulting in over three years of missing critical security improvements.

Recommendation

Upgrade all dependencies to the latest stable versions

Status

Resolved



Medium

[M-01] WalletConnect v1 Deprecation - Sunset protocol causing complete wallet connection failure

Description

The application uses the deprecated WalletConnect v1 protocol through wagmi@0.1.2, which was officially sunset on June 28, 2023. This causes wallet connection failures and exposes users to deprecated protocol vulnerabilities.

Vulnerability Details

The application imports and uses the deprecated WalletConnect v1 connector:

```
npm      ls      @walletconnect/client      @walletconnect/legacy-client      |      tee
security-repro/results/H-02/walletconnect-versions.txt

nextjs-wagmi@      /Users/lexilominite/Documents.bak/HashLock/LSA/ReGO      -
Warp/lisa-frontend-master 4

└─ wagmi@0.1.2
  └─ @walletconnect/ethereum-provider@1.7.0
    └─ @walletconnect/client@1.7.0
      └─ @walletconnect/signer-connection@1.7.0
        └─ @walletconnect/client@1.7.0 deduped
```

Impact

- Core Functionality Failure: Wallet connections fail completely as wallets disable v1 support
- User Experience: Complete inability to connect wallets = zero app functionality
- Business Impact: Application becomes unusable for a significant portion of users
- Future Risk: Problem worsens as more wallets drop v1 support

Recommendation

Upgrade to WalletConnect v2 by updating wagmi to the latest version.



Status

Resolved

[M-02] Complete Absence of Security Header - Critical web application security protections are missing

Description

The LSA Frontend application has ZERO security header protection and is completely vulnerable to common web attacks. Comprehensive deployment testing revealed that the application fails all modern web security standards.

Vulnerability Details

The application does not contain the following Security Headers:

1. Content-Security-Policy: MISSING - Zero XSS protection
2. X-Frame-Options: MISSING - Complete clickjacking vulnerability
3. X-Content-Type-Options: MISSING - MIME sniffing attacks possible
4. Referrer-Policy: MISSING - Information leakage via referrer headers
5. Strict-Transport-Security: MISSING - HTTPS downgrade attacks
6. X-XSS-Protection: MISSING - Legacy XSS protection disabled
7. Permissions-Policy: MISSING - Uncontrolled browser feature access
8. Cache-Control: MISSING - Uncontrolled caching behavior

Impact

The application is open to the following risks:

- 1) Wallet Hijacking: Clickjacking allows malicious iframes to trick users into connecting wallets to attacker sites, leading to complete fund theft.
- 2) Cross-Site Scripting (XSS): Lack of Content Security Policy enables malicious scripts to steal user data (localStorage, sessionStorage, wallet data) and execute arbitrary JavaScript.
- 3) Information Disclosure: "X-Powered-By" header reveals Next.js, and no referrer policy aids reconnaissance and privacy violations, increasing the attack surface.

- 4) MIME Sniffing: No "X-Content-Type-Options" allows files to be misinterpreted, creating XSS vectors.
- 5) Cache Poisoning: Absence of Cache-Control headers can lead to sensitive data caching, data leakage, and content integrity issues.

Recommendation

It is highly recommended to implement all these security headers in next.config.js

Status

Resolved

Conclusion

After Hashlock's analysis, the Rubicon project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.